MODUL 12 · BAB IV · DYNAMIC PROGRAMMING

Longest Common Subsequence (LCS)

Mencari subbarisan persekutuan terpanjang dari dua barisan dengan tabel Dynamic Programming dan menelusurinya kembali dengan PRINT-LCS.

TUJUAN PEMBELAJARAN

Setelah mempelajari modul ini, kamu diharapkan mampu:

- Membedakan subbarisan (*subsequence*) dengan substring dan menentukan subbarisan persekutuan dua barisan.
- Menurunkan rumus rekursif $c[i, j]$ untuk panjang LCS.
- Mengisi tabel c dan b dengan LCS-LENGTH dan menjelaskan arti setiap tanda panah.
- Menelusuri PRINT-LCS untuk memperoleh LCS dari tabel b.
- Menjelaskan running time $\Theta(mn)$ dan mengapa LCS bisa lebih dari satu.

PRASYARAT

- Modul 10: konsep dan empat langkah Dynamic Programming.
- Modul 02: rekursi (untuk PRINT-LCS).

BAB / MATERI ASAL

Materi 9 DYNAMIC PROGRAMMING Longest Common Subsequence

POSISI DALAM URUTAN BELAJAR

Penerapan kedua Dynamic Programming setelah Modul 11. Menutup Bab IV.

DAFTAR ISI

1. Seberapa Mirip Dua Barisan? 1
2. Subbarisan dan Subbarisan Persekutuan 2
3. Penyelesaian dengan Dynamic Programming 2
4. Contoh: X = ⟨A,B,C,B,D,A,B⟩ dan Y = ⟨B,D,C,A,B,A⟩ 4
5. Analisis 7
6. Kesimpulan 7

◆ Kuis Pemahaman 9
◆ Kunci Jawaban dan Pembahasan 10
◆ Referensi 11

PENDAHULUAN

1. Seberapa Mirip Dua Barisan?

Ahli biologi membandingkan untaian DNA, misalnya `ACCGGTCGAG...` dan `GTCGTTCGGA...`, untuk mengetahui seberapa dekat dua organisme. Program pembanding berkas (seperti perintah *diff*) mencari baris-baris yang tetap sama antara dua versi dokumen. Keduanya membutuhkan cara mengukur kemiripan dua barisan.

Salah satu ukuran yang paling alami adalah **subbarisan persekutuan terpanjang** (*Longest Common Subsequence*, LCS): barisan terpanjang yang muncul di kedua barisan dengan urutan yang sama, walaupun tidak harus bersebelahan. Semakin panjang LCS-nya, semakin mirip kedua barisan.

Mencari LCS dengan mencoba semua kemungkinan membutuhkan waktu eksponensial. Modul ini menerapkan keempat langkah Dynamic Programming (Modul 10) untuk menyelesaikannya dalam waktu $\Theta(mn)$, lalu menelusuri contoh dari bahan kuliah: $X = \langle A, B, C, B, D, A, B \rangle$ dan $Y = \langle B, D, C, A, B, A \rangle$.

PEMBAHASAN INTI

2. Subbarisan dan Subbarisan Persekutuan

DEFINISI

Barisan $Z = \langle z_1, \ldots, z_k \rangle$ adalah **subbarisan** (*subsequence*) dari $X = \langle x_1, \ldots, x_m \rangle$ jika Z dapat diperoleh dengan **menghapus nol atau lebih elemen** dari X tanpa mengubah urutan elemen yang tersisa.

Z adalah **subbarisan persekutuan** (*common subsequence*) dari X dan Y jika Z subbarisan dari X **dan** dari Y. **LCS** adalah subbarisan persekutuan dengan panjang maksimum.

Contoh: $\langle B, C, D, B \rangle$ adalah subbarisan dari $\langle A, \underline{B}, \underline{C}, B, \underline{D}, A, \underline{B} \rangle$ (elemen ke-2, 3, 5, 7). Perhatikan bahwa subbarisan **tidak harus bersebelahan**. Inilah bedanya dengan *substring*, yang harus berupa potongan berurutan.

2.1 Pencarian secara brute force

Bahan kuliah mendaftar beberapa subbarisan persekutuan dari $X = \langle A, B, C, B, D, A, B \rangle = \langle x_1, \ldots, x_7 \rangle$ dan $Y = \langle B, D, C, A, B, A \rangle = \langle y_1, \ldots, y_6 \rangle$:

Subbarisan persekutuan	Posisi di X	Posisi di Y	Panjang
$\langle A, B, A \rangle$	x_1, x_2, x_6	y_4, y_5, y_6	3
$\langle B, C, B, A \rangle$	x_2, x_3, x_4, x_6	y_1, y_3, y_5, y_6	4
$\langle C, B, A \rangle$	x_3, x_4, x_6	y_3, y_5, y_6	3
$\langle B, D, A, B \rangle$	x_4, x_5, x_6, x_7	y_1, y_2, y_4, y_5	4
$\langle D, A, B \rangle$	x_5, x_6, x_7	y_2, y_4, y_5	3
$\langle A, B \rangle$	x_6, x_7	y_4, y_5	2
$\langle B \rangle$	x_7	y_1	1

Tabel 1. Beberapa subbarisan persekutuan X dan Y (dari bahan kuliah).

LCS-nya adalah $\langle B, C, B, A \rangle$ dan $\langle B, D, A, B \rangle$, keduanya sepanjang 4. Jadi **LCS tidak selalu tunggal**. Masalahnya, cara ini tidak sistematis: X sepanjang m memiliki 2^m subbarisan (setiap elemen boleh diambil atau tidak), sehingga memeriksa semuanya membutuhkan waktu eksponensial.

PEMBAHASAN INTI

3. Penyelesaian dengan Dynamic Programming

Notasi: **prefiks** ke-i dari X adalah $X_i = \langle x_1, \ldots, x_i \rangle$. Misalnya untuk $X = \langle A, B, C, B, D, A, B \rangle$, $X_4 = \langle A, B, C, B \rangle$ dan X_0 adalah barisan kosong.

3.1 Langkah 1: struktur solusi optimal

Perhatikan elemen terakhir kedua barisan, x_m dan y_n (Cormen dkk., 2009, Teorema 15.1):

- Jika $x_m = y_n$, elemen itu pasti menjadi elemen terakhir suatu LCS, dan sisanya adalah LCS dari X_{m-1} dan Y_{n-1}.
- Jika $x_m \neq y_n$, paling tidak salah satu dari x_m atau y_n tidak dipakai. Maka LCS adalah yang lebih panjang di antara LCS(X_{m-1}, Y) dan LCS(X, Y_{n-1}).

Dalam kedua kasus, LCS tersusun dari LCS prefiks yang lebih pendek: **optimal substructure**. Submasalahnya juga **tumpang tindih**: LCS(X_{m-1}, Y_{n-1}) dibutuhkan oleh LCS(X_{m-1}, Y) maupun LCS(X, Y_{n-1}).

3.2 Langkah 2: rumus rekursif

Misalkan $c[i, j]$ adalah panjang LCS dari X_i dan Y_j. Maka

$$c[i,j] = \begin{cases} 0, & \text{jika } i = 0 \text{ atau } j = 0, \\ c[i-1, j-1] + 1, & \text{jika } i, j > 0 \text{ dan } x_i = y_j, \\ \max\big(c[i, j-1],\ c[i-1, j]\big), & \text{jika } i, j > 0 \text{ dan } x_i \neq y_j. \end{cases}$$

Panjang LCS dari X dan Y seluruhnya adalah $c[m, n]$.

3.3 Langkah 3: LCS-LENGTH

Tabel $c[0..m, 0..n]$ menyimpan panjang LCS, dan tabel $b[1..m, 1..n]$ menyimpan **arah** asal nilai $c[i, j]$, yang nanti dipakai untuk menyusun LCS.

LCS-LENGTH(X, Y)

```
1  m = X.length
2  n = Y.length
3  let b[1..m, 1..n] and c[0..m, 0..n] be new tables
4  for i = 1 to m
5      c[i, 0] = 0
6  for j = 0 to n
7      c[0, j] = 0
8  for i = 1 to m
9      for j = 1 to n
10         if x[i] == y[j]
11             c[i, j] = c[i-1, j-1] + 1
12             b[i, j] = "↖"
13         elseif c[i-1, j] ≥ c[i, j-1]
14             c[i, j] = c[i-1, j]
15             b[i, j] = "↑"
16         else c[i, j] = c[i, j-1]
17             b[i, j] = "←"
18 return c and b
```

- **Langkah 4–5**: kolom 0 tabel c (baris 1 sampai m) diisi 0.
- **Langkah 6–7**: baris 0 tabel c (kolom 0 sampai n) diisi 0. Keduanya menyatakan LCS dengan barisan kosong berpanjang 0.
- **Langkah 8–17**: mengisi $c[i, j]$ dan $b[i, j]$ baris demi baris, kiri ke kanan. Ada tiga kemungkinan:

Kondisi	Isi $c[i,j]$	Isi $b[i,j]$	Arti
Langkah 10 benar: $x_i = y_j$	$c[i-1,j-1]+1$ (langkah 11)	"↖" (langkah 12)	Elemen x_i menjadi bagian LCS; lanjut dari sel diagonal kiri atas
Langkah 13 benar: $c[i-1,j] \geq c[i,j-1]$	$c[i-1,j]$ (langkah 14)	"↑" (langkah 15)	Buang x_i; lanjut dari sel di atas
Selain itu	$c[i,j-1]$ (langkah 16)	"←" (langkah 17)	Buang y_j; lanjut dari sel di kiri

Tabel 2. Tiga kemungkinan pengisian sel $c[i,j]$ dan $b[i,j]$.

CATATAN KOREKSI TERHADAP SUMBER

Pada uraian langkah 11, bahan kuliah menulis rumus "$c[i,j] = c[i-1,j] + c[i,j-1] + 1$". Rumus itu **keliru**; kalimat penjelasannya sendiri sudah benar, yaitu "isi tabel c pada baris $i-1$ kolom $j-1$ ditambah 1". Rumus yang benar, sesuai pseudocode: $c[i,j] = c[i-1,j-1] + 1$. Sebagai contoh, $c[2,1]$ ($x_2 = y_1 = B$) bernilai $c[1,0]+1 = 1$; rumus yang keliru akan memberi $c[1,1]+c[2,0]+1 = 0+0+1$, yang kebetulan sama, tetapi untuk $c[4,5]$ ($x_4 = y_5 = B$) rumus keliru memberi $c[3,5]+c[4,4]+1 = 2+2+1 = 5$, padahal nilai yang benar adalah $c[3,4]+1 = 3$.

CONTOH DAN PENERAPAN

4. Contoh: X = ⟨A,B,C,B,D,A,B⟩ dan Y = ⟨B,D,C,A,B,A⟩

4.1 Tabel c dan b

Setiap sel berisi nilai $c[i,j]$ dan tanda panah $b[i,j]$. Sel berwarna kuning adalah jalur yang dilalui PRINT-LCS; sel biru adalah sel "↖" pada jalur itu, yaitu elemen yang dicetak sebagai LCS.

	j	0	1	2	3	4	5	6
i		y_j	B	D	C	A	B	A
0	x_i	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	1 ↖	1 ←	1 ↖
2	B	0	**1 ↖**	1 ←	1 ←	1 ↑	2 ↖	2 ←
3	C	0	1 ↑	1 ↑	**2 ↖**	2 ←	2 ↑	2 ↑
4	B	0	1 ↖	1 ↑	2 ↑	2 ↑	**3 ↖**	3 ←
5	D	0	1 ↑	2 ↖	2 ↑	2 ↑	3 ↑	3 ↑
6	A	0	1 ↑	2 ↑	2 ↑	3 ↖	3 ↑	**4 ↖**
7	B	0	1 ↖	2 ↑	2 ↑	3 ↑	4 ↖	4 ↑

Tabel 3. Tabel c dan b hasil LCS-LENGTH untuk X = ⟨A,B,C,B,D,A,B⟩ dan Y = ⟨B,D,C,A,B,A⟩.

Panjang LCS adalah $c[7,6] = 4$.

4.2 Menghitung beberapa sel

- $c[1,4]$: $x_1 = A = y_4$, maka $c[1,4] = c[0,3] + 1 = 1$, $b = \nwarrow$.
- $c[1,5]$: $x_1 = A \neq B = y_5$. Bandingkan $c[0,5] = 0$ dengan $c[1,4] = 1$. Karena $0 \geq 1$ salah, $c[1,5] = c[1,4] = 1$, $b = \leftarrow$.
- $c[2,4]$: $x_2 = B \neq A = y_4$. $c[1,4] = 1 \geq c[2,3] = 1$ benar (sama besar), maka $c[2,4] = c[1,4] = 1$, $b = \uparrow$. Perhatikan: ketika sama besar, LCS-LENGTH memilih "↑".
- $c[4,5]$: $x_4 = B = y_5$, maka $c[4,5] = c[3,4] + 1 = 2 + 1 = 3$, $b = \nwarrow$.
- $c[6,6]$: $x_6 = A = y_6$, maka $c[6,6] = c[5,5] + 1 = 3 + 1 = 4$, $b = \nwarrow$.

4.3 Langkah 4: PRINT-LCS

Untuk memperoleh LCS-nya, mulai dari sel $b[m,n]$ dan ikuti tanda panah sampai mencapai baris 0 atau kolom 0. Setiap kali melewati "↖", elemen x_i termasuk dalam LCS. Karena penelusuran berjalan **mundur** dari akhir, elemen-elemen ditemukan dalam urutan terbalik; PRINT-LCS mengatasinya dengan memanggil dirinya sendiri **sebelum** mencetak.

PRINT-LCS(b, X, i, j)

```
1  if i == 0 or j == 0
2      return
3  if b[i, j] == "↖"
4      PRINT-LCS(b, X, i - 1, j - 1)
5      print x[i]
6  elseif b[i, j] == "↑"
7      PRINT-LCS(b, X, i - 1, j)
8  else PRINT-LCS(b, X, i, j - 1)
```

Pemanggilan awal: `PRINT-LCS(b, X, X.length, Y.length)`, yaitu `PRINT-LCS(b, X, 7, 6)`.

Penelusuran PRINT-LCS(b, X, 7, 6)

```
PRINT-LCS(b,X,7,6): b[7,6] = "↑"  → panggil (6,6)
 PRINT-LCS(b,X,6,6): b[6,6] = "↖"  → panggil (5,5), lalu cetak x6
  PRINT-LCS(b,X,5,5): b[5,5] = "↑"  → panggil (4,5)
   PRINT-LCS(b,X,4,5): b[4,5] = "↖"  → panggil (3,4), lalu cetak x4
    PRINT-LCS(b,X,3,4): b[3,4] = "←"  → panggil (3,3)
     PRINT-LCS(b,X,3,3): b[3,3] = "↖"  → panggil (2,2), lalu cetak x3
      PRINT-LCS(b,X,2,2): b[2,2] = "←"  → panggil (2,1)
       PRINT-LCS(b,X,2,1): b[2,1] = "↖"  → panggil (1,0), lalu cetak x2
        PRINT-LCS(b,X,1,0): j == 0 → return
       ← cetak x2 = B
     ← cetak x3 = C
   ← cetak x4 = B
 ← cetak x6 = A
Hasil cetak berurutan: B C B A
```

LCS yang diperoleh adalah $\langle B, C, B, A \rangle$, sesuai bahan kuliah.

CATATAN KOREKSI TERHADAP SUMBER

Bahan kuliah menyebut bahwa proses kembali ke pemanggil "berhenti sampai didapat fungsi yang memanggil adalah PRINT-LCS(b,X,8,9)". Pemanggilan pertama yang benar adalah **PRINT-LCS(b,X,7,6)**, karena X berpanjang 7 dan Y berpanjang 6 (indeks 8 dan 9 berada di luar tabel). Penelusuran langkah demi langkah di bahan kuliah sendiri memang dimulai dari (7,6).

4.4 Mengapa hasilnya BCBA, bukan BDAB?

Kedua barisan sama-sama LCS. Yang menentukan mana yang ditemukan adalah aturan ketika $c[i-1,j] = c[i,j-1]$. LCS-LENGTH memilih "↑" (langkah 13 memakai $\geq$). Jika aturannya diubah sehingga ketika sama besar dipilih "←" (memakai $>$), penelusuran dari (7,6) mengambil jalur lain dan menghasilkan $\langle B, D, A, B \rangle$. Panjangnya tetap 4. Dynamic Programming menjamin **panjang** yang optimal; LCS yang dicetak hanya salah satu dari solusi optimal.

4.5 Implementasi Python

Python: LCS-LENGTH dan PRINT-LCS

```python
def lcs_length(X, Y, utamakan_atas=True):
    m, n = len(X), len(Y)
    c = [[0] * (n + 1) for _ in range(m + 1)]
    b = [[""] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            atas, kiri = c[i - 1][j], c[i][j - 1]
            pilih_atas = atas >= kiri if utamakan_atas else atas > kiri
            if X[i - 1] == Y[j - 1]:                # x_i == y_j
                c[i][j], b[i][j] = c[i - 1][j - 1] + 1, "D"   # D = diagonal
            elif pilih_atas:
                c[i][j], b[i][j] = c[i - 1][j], "U"           # U = atas
            else:
                c[i][j], b[i][j] = c[i][j - 1], "L"           # L = kiri
    return c, b

def print_lcs(b, X, i, j):
    if i == 0 or j == 0:
        return ""
    if b[i][j] == "D":
        return print_lcs(b, X, i - 1, j - 1) + X[i - 1]
    if b[i][j] == "U":
        return print_lcs(b, X, i - 1, j)
    return print_lcs(b, X, i, j - 1)

X, Y = "ABCBDAB", "BDCABA"
for atas in (True, False):
    c, b = lcs_length(X, Y, atas)
    print(c[len(X)][len(Y)], print_lcs(b, X, len(X), len(Y)))
c, b = lcs_length("AGGTAB", "GXTXAYB")
print(c[6][7], print_lcs(b, "AGGTAB", 6, 7))
```

Keluaran program

```
4 BCBA
4 BDAB
4 GTAB
```

ANALISIS DAN HUBUNGAN KONSEP

5. Analisis

5.1 Running time dan memori

- **LCS-LENGTH**: dua perulangan bersarang mengisi $m \times n$ sel, masing-masing dalam waktu konstan (paling banyak dua perbandingan). Running time-nya $\Theta(mn)$. Memori untuk tabel b dan c juga $\Theta(mn)$.
- **PRINT-LCS**: setiap pemanggilan mengurangi i, j, atau keduanya, sehingga paling banyak $m + n$ pemanggilan: $O(m + n)$.
- Dibandingkan brute force yang memeriksa 2^m subbarisan, ini perbedaan antara eksponensial dan polinomial (Modul 05).

Jika hanya panjang LCS yang dibutuhkan, memori dapat dihemat: menghitung baris ke-i hanya memerlukan baris ke-$(i-1)$, sehingga cukup menyimpan dua baris tabel c, yaitu $O(\min(m, n))$ dengan memilih barisan yang lebih pendek sebagai kolom (Cormen dkk., 2009). Namun tanpa tabel lengkap, LCS-nya tidak dapat disusun dengan PRINT-LCS.

5.2 Perbandingan dengan Perkalian Rantai Matriks

Aspek	Rantai matriks (Modul 11)	LCS
Jenis optimasi	Minimum (operasi perkalian)	Maksimum (panjang subbarisan)
Submasalah	Sub-rantai $A_i \cdots A_j$	Pasangan prefiks X_i, Y_j
Tabel nilai / tabel pilihan	m / s	c / b
Pilihan per submasalah	$j - i$ letak pemecah	Konstan (3 kemungkinan)
Running time	$\Theta(n^3)$	$\Theta(mn)$
Penyusunan solusi	PRINT-OPTIMAL-PARENS	PRINT-LCS

Tabel 4. Dua penerapan Dynamic Programming dari bahan kuliah.

Pola keduanya sama persis dengan keempat langkah di Modul 10. Pada Bab V berikutnya, kita akan melihat masalah optimasi yang dapat diselesaikan **tanpa** mencoba semua pilihan, cukup dengan memilih yang terlihat terbaik: algoritma Greedy.

KESIMPULAN

6. Kesimpulan

- Subbarisan diperoleh dengan menghapus elemen tanpa mengubah urutan; LCS adalah subbarisan persekutuan terpanjang dan bisa lebih dari satu.
- Rumus rekursif: $c[i, j] = 0$ bila $i = 0$ atau $j = 0$; $c[i-1, j-1] + 1$ bila $x_i = y_j$; $\max(c[i, j-1], c[i-1, j])$ bila $x_i \neq y_j$.
- LCS-LENGTH mengisi tabel c (panjang) dan b (arah ↖, ↑, ←) dalam waktu $\Theta(mn)$.
- PRINT-LCS menelusuri tabel b dari (m, n) dan mencetak x_i setiap melewati ↖, dalam waktu $O(m + n)$.
- Untuk $X = \langle A, B, C, B, D, A, B \rangle$ dan $Y = \langle B, D, C, A, B, A \rangle$, panjang LCS adalah 4; PRINT-LCS menghasilkan $\langle B, C, B, A \rangle$.

Kuis Pemahaman

Kerjakan semua soal terlebih dahulu tanpa melihat kunci jawaban. Kunci jawaban beserta pembahasannya ada di halaman berikutnya.

1 **PILIHAN GANDA**

Manakah yang merupakan subbarisan dari $X = \langle K, O, M, P, U, T, E, R \rangle$?

A. $\langle K, P, O, T \rangle$
B. $\langle O, P, T, R \rangle$
C. $\langle M, O, U, R \rangle$
D. $\langle T, U, E, R \rangle$

2 **PERHITUNGAN**

Isi tabel c lengkap untuk $X = \langle A, B, C, D, E \rangle$ dan $Y = \langle A, C, E \rangle$. Berapa panjang LCS dan apa LCS-nya?

3 **PERHITUNGAN**

Pada tabel contoh di modul ini, hitung $c[5, 2]$ dan $b[5, 2]$ menggunakan rumus rekursif, dengan menunjukkan sel-sel yang dipakai.

4 **BENAR/SALAH DENGAN ALASAN**

"Karena LCS-LENGTH memakai Dynamic Programming, LCS yang dihasilkannya selalu tunggal." Benar atau salah?

5 **ANALISIS**

Mengapa PRINT-LCS memanggil dirinya sendiri **sebelum** perintah `print x[i]`, bukan sesudahnya? Apa yang terjadi jika urutannya dibalik?

6 **PERHITUNGAN**

Tentukan panjang LCS dari $X =$ "KASUR" dan $Y =$ "RUSAK". Jelaskan mengapa hasilnya kecil walaupun kedua kata memakai huruf yang sama.

7 **PERHITUNGAN**

Dua untaian DNA masing-masing sepanjang 1.000 dan 2.000 basa. Berapa banyak sel tabel c yang diisi LCS-LENGTH? Bandingkan dengan banyaknya subbarisan untaian pertama yang harus diperiksa secara brute force.

8 **PILIHAN GANDA**

Tanda "↑" pada $b[i, j]$ berarti ...

A. $x_i = y_j$ dan x_i termasuk LCS.
B. $x_i \neq y_j$ dan nilai $c[i, j]$ diambil dari $c[i-1, j]$; elemen x_i tidak dipakai.
C. $x_i \neq y_j$ dan nilai $c[i, j]$ diambil dari $c[i, j-1]$.
D. Penelusuran PRINT-LCS berhenti.

Kunci Jawaban dan Pembahasan

Bandingkan jawabanmu, lalu baca pembahasannya untuk memahami alasan di balik setiap jawaban.

Soal 1

Jawaban: B. O (ke-2), P (ke-4), T (ke-6), R (ke-8) muncul dengan urutan yang sama. A salah karena P muncul setelah O di X, bukan sebelumnya. C salah karena O muncul sebelum M. D salah karena U muncul sebelum T.

Soal 2

		0	A	C	E
0		0	0	0	0
1	A	0	1 ↖	1 ←	1 ←
2	B	0	1 ↑	1 ↑	1 ↑
3	C	0	1 ↑	2 ↖	2 ←
4	D	0	1 ↑	2 ↑	2 ↑
5	E	0	1 ↑	2 ↑	3 ↖

Panjang LCS = $c[5, 3] = 3$, dan LCS-nya $\langle A, C, E \rangle$ (seluruh Y adalah subbarisan dari X).

Soal 3

$x_5 = D = y_2$, sehingga $c[5, 2] = c[4, 1] + 1 = 1 + 1 = 2$ dan $b[5, 2] = \nwarrow$. Sel yang dipakai hanya sel diagonal $c[4, 1]$.

Soal 4

Salah. Bisa ada beberapa LCS dengan panjang yang sama, misalnya BCBA dan BDAB pada contoh modul ini. LCS-LENGTH menjamin panjang yang maksimum; aturan pemilihan ketika nilai sama (↑ atau ←) menentukan LCS mana yang dicetak.

Soal 5

PRINT-LCS menelusuri dari belakang ($i = m$) ke depan. Dengan memanggil rekursi lebih dulu, pencetakan ditunda sampai semua elemen yang lebih awal sudah dicetak, sehingga LCS tercetak dalam urutan yang benar (dari depan ke belakang). Jika urutannya dibalik, LCS tercetak terbalik, yaitu "ABCB" untuk contoh di modul ini.

Soal 6

Panjang LCS = **1** (misalnya "K" saja). Walaupun hurufnya sama, urutannya terbalik: "RUSAK" adalah kebalikan "KASUR". Subbarisan persekutuan harus mempertahankan urutan, dan tidak ada dua huruf berbeda yang urutannya sama di kedua kata.

Soal 7

Banyaknya sel yang dihitung $1.000 \times 2.000 = 2.000.000$ (tabel lengkap berukuran 1.001×2.001 termasuk baris dan kolom 0). Brute force harus memeriksa $2^{1000} \approx 10^{301}$ subbarisan dari untaian pertama, angka yang mustahil dikerjakan.

Soal 8

Jawaban: B. Pilihan A menggambarkan "↖", pilihan C menggambarkan "←".

Referensi

Sumber utama: Materi 9 DYNAMIC PROGRAMMING Longest Common Subsequence (bahan kuliah Perancangan dan Analisis Algoritma). Penjelasan tambahan, koreksi, dan pengayaan mengacu pada pustaka berikut.

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). Cambridge, MA: MIT Press. ISBN 978-0-262-03384-8.
2. Kleinberg, J., & Tardos, É. (2006). *Algorithm Design*. Boston: Pearson/Addison-Wesley.